

课程笔记

最优传输的理论与计算系列讲座之九：几何变分算法

顾险峰（主讲） 课程笔记整理

2024 年



视频作者/频道: Upupupxiaohua (Bilibili)

发布日期: 系列课程

视频时长: 84 分钟

视频链接: <https://www.bilibili.com/video/BV1Da4y1U78N?p=9>

目录

1 引言：从理论到计算的过渡

1.1 课程背景与目标

本讲座是顾险峰教授“最优传输的理论与计算”系列课程的第九讲，主题为**几何变分算法**。在之前的八讲中，课程主要聚焦于最优传输的理论基础；从本讲开始，进入第二阶段——**计算实现阶段**。

课程两阶段划分

- **第一阶段 (第 1–8 讲)**: 最优传输的理论基础，包括 Monge-Kantorovich 问题、Brenier 定理、Minkowski 问题、球面最优传输等。
- **第二阶段 (第 9–16 讲)**: 最优传输的计算实现，包括数据结构、几何变分算法、数值方法等。

顾教授指出，许多工科背景的同学反映对理论部分“没有彻头彻尾的理解”，这很正常——“你肯定需要时间的慢慢消化，然后另外的需要真正动手。如果动手做一遍之后的话呢，你也会深刻很多。”同时，许多计算机背景的同学要求多讲算法的具体实现，因此本阶段的核心目标就是**将抽象的数学理论转化为实用的计算机算法**。

1.2 几何编程的核心挑战：数据结构

顾教授开宗明义地指出，几何编程的第一个坎就是**数据结构**：

“每个行业都有自己的看家本领。对于计算机而言，数据结构是基本功之一。那么对于几何编程，数据结构是一个很大很大的难点。”

几何编程的三座大山

学习几何编程时，通常会遇到三个层次的困难：

1. **拓扑层面**：上同调 (cohomology) 是第一个坎
2. **几何层面**：曲率、度量等概念的理解
3. **编程层面**：数据结构的设计与实现——这是今天讲座的核心

1.3 工业软件与几何引擎

顾教授将几何编程置于工业软件的大背景中讨论。工业软件的核心由三大块组成：

1. **网格生成** (Mesh Generation)
2. **数值 PDE 求解**
3. **可视化** (Visualization)

网格生成的难度

在计算力学领域，**70% 的时间都在做网格生成**。网格生成虽然是最初始的一步，但实际是最困难的一步。生成三角剖分相对容易，但生成四边形网格或六面体网格 (hex mesh) 被称为“圣杯问题” (Holy Grail)，难度极高。

2 最优传输计算的整体流程

2.1 从三维扫描到最优传输映射

在介绍数据结构之前，顾教授通过一个完整的演示展示了最优传输计算的端到端流程：

1. **输入**：通过 3D 扫描仪或 3D 相机获取三维曲面（如人脸模型）
2. **Remeshing**：对原始扫描数据进行重新网格化，得到质量较好的三角网格
3. **保角映射**：将三维曲面保角映射到二维平面（单位圆盘）上
4. **测度定义**：在圆盘上定义离散测度——每个顶点处将相邻小三角形的面积加起来除以 1/3，得到该顶点处的测度
5. **最优传输计算**：计算圆盘上该测度与 Lebesgue 测度之间的最优传输映射
6. **等价转化**：该问题等价于计算 **Brenier 势能函数**

Brenier 势能函数

最优传输映射的计算等价于求解 Brenier 势能函数 φ 。通过牛顿法迭代，通常 **4-5 步即可收敛**。收敛后得到的是 Power 图（Power Diagram），即 Voronoi 图的一种加权推广。

2.2 演示中的关键观察

从演示中可以观察到几个重要现象：

- **收敛速度快**：牛顿法迭代 4-5 步即可收敛，说明算法的数值效率很高
- **概念可视化**：抽象的数学概念（Brenier 势能、Power 图、测度）都可以通过计算机可视化
- **高维推广**：理论上，表面上的算法可以推广到高维，但”工程上极其极其复杂”

Power 图与 Voronoi 图

Power 图是 Voronoi 图的加权推广。在 Voronoi 图中，每个生成点的势力范围由欧氏距离决定；在 Power 图中，每个生成点还有一个权重，区域划分由 **Power 距离** 决定：

$$\text{Power}(x, p_i) = \|x - p_i\|^2 - w_i$$

其中 w_i 是第 i 个生成点的权重。最优传输问题的解与 Power 图有深刻的联系。

3 半边数据结构（Half-Edge Data Structure）

3.1 为什么需要半边结构

三角网格是计算机上表达曲面的标准方式。一个三角网格包含：

- **顶点**（Vertices）：包含位置坐标 (x, y, z) 、颜色、法向量等属性
- **边**（Edges）：连接两个顶点的线段

- 面 (Faces): 由三个顶点构成的三角形

三角网格表达的三层信息

1. **拓扑信息**: 单纯复形的组合结构, 表达流形的拓扑
2. **共形结构**: 每个三角形的三个内角
3. **黎曼度量**: 每条边的边长, 定义离散度量

半边结构的核心思想是: **每条边被定向为两条方向相反的半边**。这种定向使得我们可以系统地遍历网格的局部邻域。

3.2 半边结构的指针关系

半边结构以半边 (half-edge) 为核心对象, 通过指针建立各几何单元之间的关系:

表 1: 半边结构中的指针关系

对象	指针	指向
半边	next	同面内的下一条半边
半边	prev	同面内的前一条半边
半边	twin	对向的另一半边 (方向相反)
半边	vertex	该半边的终点顶点
半边	edge	该半边所属的边
半边	face	该半边所属的面
顶点	halfedge	以该顶点为终点的某条半边
边	halfedge	该边的一条半边
面	halfedge	该面内的一条半边

指针冗余与平衡

半边结构中的指针信息是有冗余的——理论上可以去掉一些指针而不丢失信息。但指针越多, 局部操作的寻址越快; 指针越少, 内存占用越小。实际实现需要在**时间复杂度**和**空间复杂度**之间找一个 balance。

3.3 半边结构的遍历操作

半边结构的核心优势在于可以高效地进行各种局部遍历:

1. **围绕面遍历**: 给定一个半边 h , 通过反复访问 $h \rightarrow \text{next}$, 可以遍历该面内的所有半边
2. **围绕顶点遍历**: 给定一个半边 h , 通过 $h \rightarrow \text{twin} \rightarrow \text{next}$, 可以围绕顶点逆时针旋转到相邻的下一个半边
3. **访问邻域**: 通过上述操作, 可以访问一个顶点的所有相邻顶点、相邻边、相邻面

欧拉操作

半边结构支持所谓的**欧拉操作**——对数据结构进行局部修改（如添加/删除顶点、边、面），同时保持拓扑一致性。这在动态网格（如流体模拟、网格自适应细化）中至关重要。

3.4 半边结构上的几何属性

为了进行几何计算，需要在半边结构的基础上定义各种属性：

表 2: 几何属性在半边结构上的分布

几何概念	定义位置	应用
边长	边 (Edge)	黎曼度量、面积计算
内角	半边 (Half-Edge)	共形结构、角度计算
高斯曲率	顶点 (Vertex)	$2\pi -$ 相邻内角和
共形因子	顶点 (Vertex)	三维面积/二维面积之比
Laplace 算子	边 (Edge)	调和映射、Poisson 方程
微分形式	半边 (Half-Edge)	上同调、外微分

3.5 C++ 实现要点

顾教授强调，几何编程的核心实现语言是 C++，原因如下：

1. **指针操作**：几何编程需要大量指针操作，C++ 在这方面最灵活
2. **性能**：所有真正的几何引擎内核都是用 C++ 编写的
3. **动态网格**：在计算过程中三角网格的组合结构会不断变化，需要精细的内存管理

Python 与 MATLAB 的局限

虽然 Python 在深度学习领域很流行，MATLAB 在数值计算中很方便，但对于几何编程：

- Python 最终也要被解释成 C，不如直接用 C++
- MATLAB 的矩阵视图难以表达动态变化的三角网格
- 指针操作和内存管理在 C++ 中最直接

4 组合映射 (Combinatorial Maps)

4.1 从半边结构到组合映射

半边结构虽然经典，但向高维推广非常困难。核心困难在于：

1. 高维情况下，指针关系变得非常复杂
2. 计算过程中更新 connectivity 时，正确性验证具有本质困难

3. 内存管理（分配/释放）容易出错

”抽象层次越高，实际上越代数化的话呢，这个思维的负担越轻，越容易被推广。当然开始理解起来就困难一点。”

4.2 组合映射的定义

组合映射（Combinatorial Maps）是一种更抽象、更代数化的数据结构，它将半边结构推广到任意维度。

组合映射的核心思想

组合映射用一组**对合变换**（involutions） $\alpha_0, \alpha_1, \dots, \alpha_n$ 来定义胞腔分解（cell decomposition）的组合结构。这些变换作用在所谓的**飞镖**（darts）集合上，生成一个自由群。

对于一个二维组合映射（对应三角网格），定义三个对合变换：

- α_0 ：在同一顶点周围的相邻飞镖之间切换
- α_1 ：在同一面上沿边的相邻飞镖之间切换
- α_2 ：在相邻两个面的对应飞镖之间切换（相当于半边结构中的 twin 操作）

表 3: 组合映射中的对合变换

变换	作用	对应半边结构
α_0	同一顶点周围的飞镖切换	围绕顶点旋转
α_1	同一面内沿边的飞镖切换	next/prev 操作
α_2	相邻面之间的飞镖切换	twin 操作

4.3 组合映射的优势

组合映射相比半边结构有以下优势：

1. **维度无关**：可以自然地推广到任意维度
2. **统一表达**：所有胞腔（顶点、边、面、体、...）及其连接关系都被统一表达
3. **寻址优化**：从一个单元到其邻域单元的寻址时间被优化
4. **内存节省**：相比半边结构，组合映射可以节省大量内存

组合映射与半边结构的关系

组合映射可以看作是半边结构的代数化抽象。半边结构中的指针关系在组合映射中被转化为群作用。这种代数化的视角虽然一开始理解起来更困难，但一旦掌握，思维的负担反而更轻，推广也更容易。

4.4 可定向性与飞镖结构

组合映射的一个关键概念是**可定向性** (orientability)。在可定向的流形上，可以定义一致的飞镖定向：

- 每条边上定义逆时针方向
- 相邻三角形在公共边上的飞镖方向相反
- 通过不断拓展，可以全局一致地定义每个半边的定向

定向的一致性

如果能全局一致地定义每个半边的定向，则称该流形是**可定向的**。对于不可定向的流形（如 Möbius 带、Klein 瓶），无法做到这一点。在实际编程中，维持定向一致性是一个常见的 bug 来源。

5 几何计算的实现框架

5.1 实现步骤概述

顾教授总结了几何计算算法的实现步骤：

1. **定义主要数据结构**：顶点、边、半边、面（以及高维情况下的 cell）
2. **定义模板参数**：使用 C++ 模板使数据结构具有通用性
3. **定义 Circulator/Iterator**：遍历相邻几何单元的迭代器
4. **定义几何操作**：计算边长、面积、角度、曲率等
5. **实现欧拉操作**：动态修改网格结构的操作
6. **求解 PDE**：最终归结为求解线性椭圆型 PDE，使用线性代数库（如 Eigen）

5.2 线性系统的求解

最优传输问题的计算最终归结为求解**对称正定线性系统**：

$$A\mathbf{x} = \mathbf{b} \tag{1}$$

其中 A 是离散的 Laplace 算子（定义在边上）， $\mathbf{x}$ 是待求的势能函数值（定义在顶点上）， $\mathbf{b}$ 是由测度差异决定的右端项。

Laplace 算子在半边结构上的定义

在三角网格上，离散的 Laplace 算子（也称 cotangent Laplacian）定义在边上：

$$L_{ij} = \begin{cases} \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}) & \text{if } i \sim j \\ -\sum_{k \sim i} L_{ik} & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

其中 α_{ij} 和 β_{ij} 是边 (i, j) 两侧的两个对角。这个算子在保角映射、调和映射、曲率流等问题中都有核心作用。

5.3 调试与工程实践

顾教授特别强调了工程实践中的几个要点：

几何编程的工程挑战

1. **内存管理**：几何编程中内存很容易出错。在微软，每段程序实际上是七个人同时在写，其中有三道工序实时检查内存操作错误。
2. **耐心**：调试几何代码需要头脑特别冷静，”如果性格急躁的话，做这个还是很困难的”。
3. **动态网格**：当网格的组合结构在计算过程中不断变化时，维持数据结构的 consistency 具有本质困难。
4. **理论与实现的鸿沟**：计算几何领域绝大多数算法论文发表后”根本没办法实现”，只是理论探讨。工业界需要实打实的实现。

6 总结与延伸

6.1 本章小结

本讲作为最优传输计算系列的第一讲，重点介绍了**几何变分算法的数据结构基础**：

1. **最优传输计算流程**：从三维扫描到保角映射，再到 Brenier 势能函数的计算，展示了完整的端到端流程
2. **半边数据结构**：几何编程的经典数据结构，通过指针建立顶点、边、半边、面之间的关系，支持高效的局部遍历和欧拉操作
3. **组合映射**：半边结构的代数化抽象，用对合变换定义胞腔分解，可以自然地推广到任意维度
4. **实现要点**：C++ 是几何编程的首选语言，需要精细的内存管理和耐心调试

6.2 核心要点回顾

本讲核心要点

1. 几何编程的第一个坎是**数据结构**，不是算法
2. 半边结构是二维曲面几何计算的经典选择
3. 组合映射提供了向高维推广的理论框架
4. 最优传输计算最终归结为求解 **Poisson 方程**（对称正定线性系统）
5. 工程实现需要大量调试，理论与实践之间存在显著鸿沟

6.3 拓展阅读

- **半边结构经典文献**：Kettner, L. *Using Generic Programming for Designing a Data Structure for Polyhedral Surfaces*. CGAL 文档
- **组合映射**：Lienhardt, P. *N-dimensional Generalized Combinatorial Maps and Cellular Quasi-manifolds*
- **离散微分几何**：Crane, K. *Discrete Differential Geometry: An Applied Introduction*
- **最优传输计算**：本系列课程的前八讲（理论基础）和后续讲座（算法实现细节）

6.4 从理论到实践的桥梁

顾教授在讲座结尾强调：

”有了计算机之后，所有抽象的数学其实都可以真正的变成实用的算法。”

这正是本系列课程第二阶段的核心目标——将最优传输的深刻数学理论（Brenier 定理、Monge-Ampere 方程、流体力学观点等）转化为可以在计算机上实际运行的算法。数据结构是这个转化过程的第一块基石。

后续课程预告

后续讲座将继续深入：

- 第 10 讲：几何变分算法（续）——更详细的算法实现
- 第 11–12 讲：球面最优传输的计算
- 第 13–14 讲：最优传输的奇异结构
- 第 15 讲：最优部分传输
- 第 16 讲：最优传输总结